Step 1: Import all libraries needed to perform methods

```
[17]: #Import from libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error
```

Step 2: Load the data set csv file

```
[18]: # Load dataset
df = pd.read_csv('breast-cancer.csv')
```

Step 3: Explore the data set and search for missing or null data

```
[19]: # Explore dataset
print(df.info())
print(df.describe())
print("Missing values per column:\n", df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
None
```

```
                 id  radius_mean  texture_mean  perimeter_mean    area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000   569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033   654.889104
std    1.250206e+08     3.524049      4.301036       24.298981   351.914129
min    8.670000e+03     6.981000      9.710000       43.790000   143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000   420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000   551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000   782.700000
max    9.113205e+08    28.110000     39.280000      188.500000  2501.000000

       smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
min           0.052630          0.019380        0.000000             0.000000
25%           0.086370          0.064920        0.029560             0.020310
50%           0.095870          0.092630        0.061540             0.033500
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200

       symmetry_mean  ...  radius_worst  texture_worst  perimeter_worst  \
count     569.000000  ...    569.000000     569.000000       569.000000
mean        0.181162  ...     16.269190      25.677223       107.261213
std         0.027414  ...      4.833242       6.146258        33.602542
min         0.106000  ...      7.930000      12.020000        50.410000
25%         0.161900  ...     13.010000      21.080000        84.110000
50%         0.179200  ...     14.970000      25.410000        97.660000
75%         0.195700  ...     18.790000      29.720000       125.400000
max         0.304000  ...     36.040000      49.540000       251.200000

        area_worst  smoothness_worst  compactness_worst  concavity_worst  \
count   569.000000        569.000000         569.000000       569.000000
mean    880.583128          0.132369           0.254265         0.272188
std     569.356993          0.022832           0.157336         0.208624
min     185.200000          0.071170           0.027290         0.000000
25%     515.300000          0.116600           0.147200         0.114500
50%     686.500000          0.131300           0.211900         0.226700
75%    1084.000000          0.146000           0.339100         0.382900
max    4254.000000          0.222600           1.058000         1.252000

       concave points_worst  symmetry_worst  fractal_dimension_worst
count            569.000000      569.000000               569.000000
mean               0.114606        0.290076                 0.083946
std                0.065732        0.061867                 0.018061
min                0.000000        0.156500                 0.055040
25%                0.064930        0.250400                 0.071460
50%                0.099930        0.282200                 0.080040
75%                0.161400        0.317900                 0.092080
max                0.291000        0.663800                 0.207500

[8 rows x 31 columns]
```

```
Missing values per column:
 id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

Findings: The dataset analysis reveals a comprehensive collection of 569 samples across 31 numerical features, all stored as float64 data type. Notably, there are no missing or null values in any of the columns, indicating a clean and complete dataset. The features encompass various measurements related to cancer diagnosis, including radius, texture, perimeter, area, smoothness,

compactness, concavity, symmetry, and fractal dimensions - each having meaning, standard error, and worst value variations. This clean state of the data suggests it's ready for immediate analysis or modeling without requiring any missing value handling.

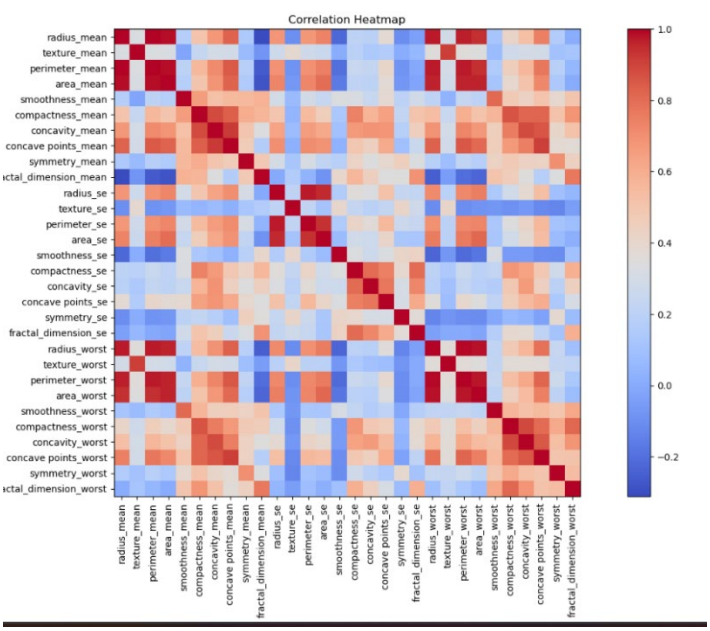
Step 4: Remove unnecessary columns

```
[20]: # Drop ID because it is ireelevant to know the annonymous patients used in the sample data
      df = df.drop(columns=['id'], errors='ignore')
```

Findings: In the previous step, we confirmed that the data was clean with no missing values. Upon analyzing the column headers, I identified that the 'id' column could be removed from the dataset since it only contains anonymous patient identification numbers that don't contribute to understanding or predicting breast cancer characteristics. These ID numbers are simply used for record-keeping and have no medical or diagnostic significance for breast cancer analysis.

Step 5: Calculate feature correlations and visualize them using a Heatmap

```
# Discover and visualize the data to gain insights

# Drop non-numeric columns for correlation calculation
numeric_df = df.select_dtypes(include=[np.number])

# Calculate correlation matrix for numeric columns only
corr_matrix = numeric_df.corr()

# Plot the correlation heatmap
plt.figure(figsize=(18, 9))
plt.imshow(corr_matrix, cmap='coolwarm', interpolation='nearest')
plt.colorbar()
plt.title("Correlation Heatmap")

# Add these lines to show feature names
plt.xticks(range(len(numeric_df.columns)), numeric_df.columns, rotation=90)  # X-axis labels, rotated 90 degrees
plt.yticks(range(len(numeric_df.columns)), numeric_df.columns)  # Y-axis labels

plt.show()
```



Findings: For the first part I found that the diagnosis column had letters representing if it was benign or malignant, because corr_matrix only takes numeric for calculations I had to adjust the code to only select the columns that had number based data and ignore the diagnostic column. After that I ran the correlation and chose to show it all on a heatmap. For the correlation heatmap, it reveals strong relationships among several features, particularly along the diagonal, where each variable is perfectly correlated with itself. Features such as radius_mean, perimeter_mean, and area_mean exhibit high positive correlations, indicating they measure similar characteristics. Likewise, the worst and SE (standard error) values of these features also show strong correlations within their respective groups. In contrast, some variables, such as fractal_dimension_mean and symmetry_mean, display weaker correlations with other features, suggesting they capture different aspects of the data. The presence of both strong and weak correlations highlights potential redundancy in some features while also identifying independent variables that could be useful for further analysis.

Step 6: Set Up Data for Machine Learning Modeling

```
[24]: # Split dataset into train and test sets
train_set, test_set = train_test_split(df_imputed, test_size=0.2, random_state=42)

[25]: # Prepare data for ML
x_train = train_set.drop('radius_mean', axis=1)
y_train = train_set['radius_mean']
x_test = test_set.drop('radius_mean', axis=1)
y_test = test_set['radius_mean']
```

Step 7: Train Model using Random Forest and Cross-Validation Methods

```
[26]: # Train model using RandomForestRegressor with cross-validation
forest_reg = RandomForestRegressor(n_estimators=100, random_state=42)
cross_val_scores = cross_val_score(forest_reg, x_train, y_train, scoring='neg_mean_squared_error', cv=10)
rmse_scores = np.sqrt(-cross_val_scores)
print("Cross-Validation RMSE:", rmse_scores.mean())

Cross-Validation RMSE: 0.21544783387702887
```

Findings: The cross-validation results show that the RandomForestRegressor model achieved an average Root Mean Squared Error (RMSE) of 0.215, indicating a relatively low prediction error. Since RMSE measures the average deviation between predicted and actual values, a lower value suggests better model accuracy. The use of 10-fold cross-validation ensures that the error estimate is reliable and reduces the risk of overfitting. However, the effectiveness of this RMSE depends on the scale of the target variable, and further tuning of hyperparameters may improve performance.

Step 8: Fine Tune Using Grid Seach CV Method

```
[45]: # Fine-tune the model using GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_features': [3,5,7]
}
grid_search = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train, y_train)
print("Best Parameters:", grid_search.best_params_)

Best Parameters: {'max_features': 7, 'n_estimators': 50}
```

Findings: In this GridSearchCV process, I tested different values for n_estimators (50, 100, and 150) and max_features (3, 5, and 7) to find the optimal settings for the RandomForestRegressor. The goal was to identify the best number of trees and features to use at each split. After running a 5-fold cross-validation, the best combination of parameters turned out to be n_estimators: 50 and max_features: 7. This suggests that using 50 trees along with considering 7 features at each split gave the best performance for this dataset, balancing model accuracy and computational efficiency.

Step 9: Perform Final Model on Test Set

```
[43]: # Evaluate model on test set
final_model = grid_search.best_estimator_
y_pred = final_model.predict(x_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("Test Set RMSE:", rmse)
```

Findings: The evaluation of the final model on the test set resulted in a Root Mean Squared Error (RMSE) of 0.235. This indicates that the model's predictions on the test data are fairly close to the actual values, with an average error of around 0.235. The relatively low RMSE suggests that the model is performing well and accurately predicting the target values for the given dataset.